

TP2 • SPLN 2025/26

Pipeline de Information Retrieval

e Question Answering sobre a Wikipedia

João Moura

Universidade do Minho • Mestrado em Engenharia Informática

Maio de 2026

01 Objetivo

Construir uma pipeline que responda a perguntas em linguagem natural sobre os países do mundo, combinando retrieval de informação e question answering.



Tarefas do enunciado

- a Corpus ≥ 100 documentos sobre tema à escolha
- b Módulo retriever (léxico, semântico ou híbrido)
- c QA extractivo (fine-tuning SQuAD) + QA abstrativo (prompting LLM)

02 Corpus

Fonte e processo

- Artigos Wikipedia (EN) de 196 países — 193 Estados-membros da ONU + observers
- Biblioteca wikipedia-api (User-Agent próprio, conforme política Wikipedia)
- Filtros: See also, References, External links, Notes, Further reading
- Override manual para títulos ambíguos (Georgia → país, não estado dos EUA)
- Cache em disco — retomar em caso de interrupção sem repetir pedidos

194

países descarregados (de 196)

8 919

chunks após processamento

EN

língua do corpus (inglês)

Wikipedia oferece artigos com estrutura uniforme e cobertura factual ampla — ideal para perguntas geográficas, demográficas, históricas.

03 Pré-processamento e Chunking

Porquê chunking?

- Artigos têm dezenas de milhares de caracteres
- BERT limita o contexto a 512 tokens — passagens menores funcionam melhor
- Retriever recupera melhor sobre passagens curtas e tematicamente coerentes

Estratégia: parágrafos naturais

- Split por `\n\n` (parágrafos)
- Filtrar < 30 palavras (lixo: títulos, frases soltas)
- Output: corpus.json com {id, country, text}

```
def chunk_article(text, min_words=30):  
    paragraphs = [p.strip()  
                  for p in text.split("\n\n")  
                  if p.strip()]  
    return [p for p in paragraphs  
            if len(p.split()) >= min_words]
```

Decisões justificadas: (i) preserva coerência semântica do parágrafo (um tópico por chunk); (ii) implementação simples e reproduzível, sem hiperparâmetros sensíveis a tunar.

BM25

Léxico

- Generalização do TF-IDF
- Baseline forte e clássica em IR
- rank-bm25 (BM25Okapi)
- Tokenização: lowercase + \w+
- Sem stopwords (IDF compensa)

Sentence-BERT

Semântico

- Embeddings densos (384 dim)
- multi-qa-MiniLM-L6-cos-v1
- Treinado para QA retrieval
- Similaridade cosseno
- Embeddings em cache (.npy)

Híbrido

Reciprocal Rank Fusion

- Combina rankings, não scores
- $k = 60$ (constante padrão)
- Evita escalas diferentes
- Mais robusto na maioria dos casos

$$\text{RRF}(d) = \sum \frac{1}{(k + \text{rank}_r(d))} \quad \text{com} \quad R = \{ \text{BM25}, \text{sBERT} \} \quad \text{e} \quad k = 60$$

05 Retriever — Análise Comparativa

3 queries-teste, cada uma revela um padrão diferente

Query	BM25	Semantic	Hybrid (RRF)
Q1: Capital de Portugal (factoide, keyword óbvia)	X Neanderthals	✓ Lisbon	✓ Lisbon
Q2: Fronteiras do Brasil (relacional)	X Uruguai	✓ Brasil (com lista)	≈ Uruguai (RRF perde)
Q3: Quando reunificou a Alemanha? (paráfrase: reunify ≠ reunification)	X Camarões, Luxemburgo, M. Is.	✓ Germany (3/3)	✓ Germany (top-1 recupera)

Insights

- **BM25:** ganha em keywords exatas, mas é enganado por alta frequência em contextos errados
- **Sentence-BERT:** é superior em paráfrases e queries relacionais — entende o significado
- **Híbrido (RRF):** é robusto na maioria dos casos, mas em Q2 piora o semantic isolado (favorece o consenso entre rankings)

06 QA Extractivo

Devolve uma span do contexto que constitui a resposta

Modelo & Fine-Tuning

- distilbert-base-uncased — 40% menor, 60% mais rápido que BERT, ~97% do desempenho downstream
- Fine-tuned no SQuAD v1.1 (Rajpurkar et al. 2016)
- Subset: 20k train · 500 eval (≈30 min em T4 GPU)
- `lr=3e-5 · batch=16 · 2 épocas · fp16`

Resultados (SQuAD val, 500 ex)

69.80

Exact Match

78.52

F1 Score

(literatura: full SQuAD ≈ EM 77 · F1 85)

Integração com o retriever

1. hybrid_search → top-3 chunks 2. DistilBERT extrai span de cada 3. escolher max(qa_score)

Limitação: modelo extractivo não tem mecanismo de abstenção — devolve sempre alguma span, mesmo quando o contexto não responde à pergunta.

Retrieval-Augmented Generation: o retriever alimenta o modelo generativo

Modelo

- google/flan-t5-base — 250M parâmetros
- Variante instruction-tuned do T5
- Treinado em 1800+ tarefas formuladas como instruções
- Capacidade zero-shot robusta com prompt bem estruturado

Diferenciadores vs. extractivo

- Pode parafrasear e sintetizar várias fontes
- Pode auto-detectar "unanswerable"
- Camada de filtragem: respostas-lixo → "(no clean answer)"

Prompt

Read the context below and answer the question concisely.

Context:

Article about Portugal: <chunk text...>

Article about Brazil: <chunk text...>

Article about Germany: <chunk text...>

Question: <user query>

Answer:

08

Demo: Extractivo vs. Abstrativo

Query	Extractivo	Abstrativo	Categoria
Capital de Portugal?	Lisbon ✓	(no clean answer)	Factoide
Língua oficial de Itália?	Italian ✓	(no clean answer)	Factoide
População da Islândia?	two million ✗	(no clean answer)	Numérico
Fronteiras do Brasil?	Uruguay (parcial)	Uruguay to the south ✓	Relacional
Independência da Índia?	1947 ✓	unanswerable ✓	Histórico
Reunificação da Alemanha?	World War II ✗	(no clean answer)	Paráfrase
Moeda usada em França?	the euro ✓	franc. ✗	Paráfrase
Vencedor do Mundial 2026?	Qatar ✗	(no clean answer) ✓	Armadilha

Insight-chave: após limpeza profunda do corpus + camada de filtragem, o sistema distingue claramente "resposta" de "não consegui responder" — incluindo na pergunta-armadilha. Abstenção por engenharia, não por capacidade emergente.

Conclusões

1

Retriever: semântico > léxico

Sentence-BERT é claramente superior em paráfrases e queries relacionais. BM25 mantém-se útil para entidades nomeadas raras. RRF híbrido é robusto, mas nem sempre supera o semântico isolado.

2

Extractivo e abstrativo são complementares

Extractivo: precisão em factoides, datas, nomes próprios literais. Abstrativo: flexibilidade em perguntas relacionais e paráfrases, e — crucialmente — pode abster-se quando não sabe.

3

Retriever + QA = sinergia genuína

Retriever maximiza recall (passa candidatos prováveis); QA maximiza precision (escolhe a melhor resposta dentro deles). Separação de responsabilidades torna a arquitetura robusta.

Obrigado!